APB Protocol Verification Test Plan

1. Introduction

1.1 Objective

The objective of this verification plan is to verify the functionality and protocol compliance of the Advanced Peripheral Bus (APB) slave using a UVM-based verification environment.

The verification focuses on:

- Correct APB read and write transactions
- Protocol timing compliance
- Wait-state handling
- Error response generation
- Functional correctness
- Assertion-based protocol checking
- Functional coverage closure

2. Verification Objectives

The major verification objectives are:

ID	Verification Objective
VO-1	Verify APB write operation
VO-2	Verify APB read operation
VO-3	Verify setup to enable phase transition
VO-4	Verify wait-state handling
VO-5	Verify back-to-back transfers
VO-6	Verify invalid address handling
VO-7	Verify PSLVERR assertion behavior
VO-8	Verify signal stability during wait states
VO-9	Verify protocol compliance using assertions
VO-10	Achieve functional coverage closure

3. Design Under Test (DUT)

DUT Features

The APB slave supports:

- Read transactions
- Write transactions
- Address decoding
- Wait-state insertion
- Error response generation
- PREADY handshake mechanism

---

4. Verification Environment Architecture

The verification environment is developed using Universal Verification Methodology (UVM).

Components Used

Component	Function
Sequence Item	Defines APB transaction fields
Sequence	Generates randomized transactions
Driver	Drives transactions to DUT
Monitor	Samples DUT activity
Agent	Encapsulates driver, monitor, sequencer
Scoreboard	Checks expected vs actual data
Coverage Collector	Collects functional coverage
Assertions	Checks protocol correctness
Environment	Connects all components
Test	Controls verification scenarios

---

5. Transaction Description

APB Transaction Fields

Signal		Description	Category
reset			Randomized
PADDR		Address bus	randomized

PWDATA		Write data	randomized
PRDATA		Read data	output port
PWRITE		Read/Write control	randomized
PSEL		Slave select	not randomized
PENABLE		Enable signal	not randomized
PREADY		Transfer completion	output
PSLVERR		Error indication	output

6. Verification Methodology

The verification methodology includes:

- Directed testing
- Constrained-random testing
- Assertion-based verification
- Functional coverage-driven verification
- Scoreboard-based data checking

7. Test Scenarios

Sr.No	Test Cases	Objective	Checks	Expected Result
1	Reset Verification	Verify DUT behavior during reset.	• All outputs reset correctly • No transaction occurs during reset • Internal memory initializes correctly	• DUT enters idle state after reset
2	Write Test	Verify single APB write operation.	• Address correctly decoded • Data correctly written • No PSLVERR generated	• Write completes successfully

3	Read Test	Verify single APB read operation.	• Correct data returned on PRDATA • PREADY asserted correctly	• Read data matches expected value
4	Multiple Write Transactions	Verify multiple consecutive write transactions.	• All addresses updated correctly • No data corruption • Correct protocol timing	Write Completes successfully
5	Multiple Read Transaction	Verify multiple read transactions.	• Correct data returned for all addresses No unknown data observed	Successful Read Completes
6	Rando Read/Write Test	Verify DUT under constrained-random traffic.	• Random address accesses • Random data patterns • Random read/write sequencing	DUT behaves correctly under randomized traffic
7	Wait State Verification	Verify APB wait-state handling.	• PENABLE remains asserted • Address/control signals remain stable • Transfer completes only after PREADY = 1	Correct wait-state behavior observed
8	Invalid address Test	Verify invalid address handling.	• PSLVERR asserted correctly • No memory corruption	Transfer terminates with error response
9	Error Response Verification	Verify PSLVERR timing and behavior.	• PSLVERR asserted only during valid transfer completion • PSLVERR not asserted during idle/wait states	
10	Back to Back Transfers	Verify consecutive APB transfers without idle	• Proper SETUP and ENABLE transitions • No protocol violation	

		cycles.		
11	Boundary Address	Verify lower and upper address boundaries.	• Lowest valid address access • Highest valid address access • Correct memory operation	
12	Data Pattern Verification	Verify different data patterns.	Patterns • All zeros • All ones • Alternating bits • Random values	Correct read/write operation for all patterns

8. Scoreboard Strategy

The scoreboard uses a reference memory model implemented using associative arrays.

Write Operation

- PWDATA stored using PADDR as key

Read Operation

- PRDATA compared against stored expected value

Error Checking

- Mismatch generates UVM error

---

9. Assertion-Based Verification

Assertions are implemented to verify protocol compliance.

Category	*Assertions*
Protocol Sequencing	ass_setup_to_address, ass_penable_implies_psel, ass_complete_transfer
Wait-State Handling	ass_wait_state_hold, ass_noerror_wait
Signal Stability	ass_ctrl_stable, ass_Wdata_stable
Data Integrity	ass_Rdata_valid
Protocol Legality	ass_idle_rule, ass_ctrl_notunknown

Assertion Name	Property Checked	Purpose
ass_setup_to_address	(PSEL && !PENABLE) |=> (PSEL && PENABLE)	Verifies APB transfer correctly moves from SETUP phase to ENABLE phase in the next clock cycle. Ensures proper APB state transition.
ass_penable_implies_psel	PENABLE |-> PSEL	Ensures PENABLE is never asserted unless slave select (PSEL) is active. Detects illegal APB enable behavior.
ass_wait_state_hold	(PSEL && PENABLE && !PREADY) |=> (PENABLE && PSEL)	Verifies that during wait states, the

		master keeps the transfer active by holding PSEL and PENABLE high until slave becomes ready.
ass_complete_transfer	(PSEL && PENABLE && PREADY) |=> (!PENABLE)	Ensures transfer completion behavior is correct. After PREADY becomes high, PENABLE must deassert in next cycle.
ass_ctrl_stable	$stable({PADDR, PWRITE}) during wait state	Verifies address and control signals remain stable during active transfer while slave is inserting wait states. Prevents protocol corruption.
ass_Wdata_stable	$stable(PWDATA) during write wait state	Ensures write data remains constant during stalled write transfer (PREADY=0). Prevents data corruption during wait states.
ass_Rdata_valid	!$isunknown(PRDATA) when read completes	Verifies read data is valid and not X/Z when transfer completes. Detects invalid or uninitialized read responses.
ass_idle_rule	!PSEL |-> !PENABLE	Ensures APB remains in IDLE state correctly. PENABLE should never assert when slave is not selected.

ass_ctrl_notunknown	!$isunknown({PADDR,PWRITE,PENABLE})	Checks that key APB control signals do not contain unknown (X) values during active transfer. Improves protocol reliability.
ass_noerror_wait	(PSEL && PENABLE && !PREADY) |-> (PSLVERR==0)	Verifies error signal is not asserted during wait states. PSLVERR should only be valid during completed transfer.

10. Functional Coverage Plan

Coverage Type	Coverpoint / Cross	Bins	Purpose
Operation Coverage	PWRITE Cp_write	READ = 0	Verifies read transactions are generated and exercised.
		WRITE = 1	Verifies write transactions are generated and exercised.
---	---	---	---
Address Coverage	PADDR Cp_addr	low = [0:15]	Verifies accesses to lower address region.
		mid = [16:127]	Verifies accesses to middle address region.
		high = [128:255]	Verifies accesses to higher address region.
---	---	---	---
Data Coverage	PWDATA Cp_data	zero = 0	Verifies all-zero data pattern handling.
		random = default/others	Verifies DUT behavior for random data patterns.
		max_value = all 1s	Verifies maximum possible data value handling.
---	---	---	---
Error	PSLVERR	no_error = 0	Verifies successful transfer

Coverage	Cp_error		scenarios.
		error = 1	Verifies DUT error response behavior.
---	---	---	---
Cross Coverage	READ × Address	<READ, low>	Verifies reads from low address range.
		<READ, mid>	Verifies reads from middle address range.
		<READ, high>	Verifies reads from high address range.
	WRITE × Address	<WRITE, low>	Verifies writes to low address range.
		<WRITE, mid>	Verifies writes to middle address range.
		<WRITE, high>	Verifies writes to high address range.
---	---	---	---
Cross Coverage	READ × PSLVERR	<READ, no_error>	Verifies successful read transactions.
		<READ, error>	Verifies erroneous read transactions.
	WRITE × PSLVERR	<WRITE, no_error>	Verifies successful write transactions.
		<WRITE, error>	Verifies erroneous write transactions.

Coverage Area	*Why It Was Chosen*
Operation Coverage	Ensures both read and write protocol paths are verified.
Address Coverage	Ensures complete address-space validation across different memory regions.
Data Coverage	Validates DUT behavior for important data patterns and corner cases.
Error Coverage	Ensures error handling and invalid access scenarios are verified.
Cross Coverage	Ensures combinations of operation type, address region, and error conditions are fully exercised.

11. Coverage Goals

Coverage Type	Goal
Code Coverage	> 90%
Functional Coverage	> 95%
Assertion Coverage	100% protocol checks enabled

---

12. Debug Strategy

Debugging is performed using:

- QuestaSim waveform analysis
- UVM transaction logging
- Assertion failure tracking
- Functional coverage reports
- Scoreboard mismatch analysis

---

13. Risks and Mitigation

Risk	Mitigation

Risk	Mitigation
Random scenarios not hitting coverage bins	Add directed tests
Protocol timing bugs	Use assertions
Synchronization issues	Use clocking blocks and monitored handshakes
Unknown/X propagation	Add stability assertions

14. Expected Results

The verification environment should:

- Verify all APB protocol operations
- Detect protocol violations
- Achieve coverage closure
- Identify timing and synchronization issues
- Validate DUT functionality under constrained-random traffic

15. Project Outcome

The APB verification environment successfully:

- Verified read/write functionality
- Validated protocol timing behavior
- Implemented assertion-based protocol checking
- Achieved high functional coverage
- Detected corner-case protocol issues
- Improved debugging and verification methodology skills

16. Conclusion

A complete UVM-based APB verification environment was developed using constrained-random verification, assertions, scoreboard checking, and functional coverage methodologies.

The verification environment ensured protocol compliance, functional correctness, and effective coverage closure while improving debugging efficiency and reusable verification practices.